

TSFS12 HAND-IN EXERCISE 3

Vehicle Motion Control

August 12, 2026

1 OBJECTIVE

The objective of this hand-in exercise is to explore and implement two approaches to control of a ground vehicle, pure pursuit control and state-feedback control, with the aim to follow a planned path. It will also illustrate the effects of introducing non-linear feedback laws. A simulation environment is provided and you are expected to explore properties of your solutions and summarize them in appropriate plots.

2 PREPARATIONS BEFORE THE EXERCISE

Before doing this exercise, make sure you have read and understood the material covered in

- Lecture notes on Control of autonomous vehicles I: Ground vehicles
- Paden, Brian, et al. *"A survey of motion planning and control techniques for self-driving urban vehicles"*. IEEE Transactions on intelligent vehicles 1.1 (2016): 33-55. A good but slightly advanced text and therefore many details are included in the lecture notes.
- Coulter, R. Craig. *"Implementation of the Pure Pursuit Path Tracking Algorithm"* (1992)¹.
- This exercise description, including the appendices.

To get the background code for the exercise, get the latest files from <https://gitlab.liu.se/vehsys/tsfs12> and familiarize yourself with the provided code. Start by running the code in the lab template file `main`.

You are free to choose in which language to implement your solutions, there is skeleton code available in Matlab and Python. The Python skeleton files are available in Notebook format (`.ipynb`) (<https://jupyterlab.readthedocs.io/>) and as Python script files (`.py`) and you can choose to work with the format you prefer. Code skeletons are available in Matlab and Python, but if you prefer to make your own implementation that is also possible. In the student labs on campus, all Python packages needed are pre-installed in the virtual environment activated by

```
source /courses/tsfs12/env/bin/activate
```

¹ But don't look at figure 1, it is badly scaled which makes it difficult to understand; use figures in the lecture slides instead

3 REQUIREMENTS

The objective of the exercise is to implement vehicle controllers tracking a specified planned path, evaluating their properties, and understanding when and how they are appropriate. In particular, to explore linear design techniques and be aware of their limitations, and how non-linear techniques are useful.

To complete this exercise you should implement the following controllers

- Pure-pursuit controller
- Linear state-feedback controller with feed-forward term
- Non-linear state feedback

The exercise is examined by submitting the following on the Lisam course page.

1. Runnable code. If your code consists of several files, submit a zip-archive with your files. Do not include files for the language that you did not use.
2. A report
 - that answers the questions, including relevant plots, in Section 5, and other questions that you have encountered during solving this exercise
 - important to include your reflections in the document, and not just report results and code. It is necessary to include a discussion whenever requested in the questions.
 - includes a concluding discussion

The report is expected to be a neat, self-contained, document in PDF format. You are encouraged, but not required, to use the templates provided at the course git-repo.

It is allowed to discuss the exercises on a general level with other course participants. However, code sharing outside groups is not allowed. Moreover, both students in the group should be fully involved in performing all exercises, and thereby be prepared to answer questions on all subtasks.

See Appendix D for the extra assignment needed for higher grades. The extra assignment is done individually and is submitted with a document answering the questions, including suitable figures, plots, and tables. There is only pass/fail on the extra exercise and the document can not be revised or extended after submission. It is not required to get everything correct to pass the assignment, we assess the whole submission. You are of course welcome to ask us if you have questions or want us to clarify the exercise.

Use of AI Tools

AI tools (ChatGPT, Claude, Copilot, etc.) are allowed for this course, including hand-in exercises. Use them to get unstuck, explore ideas, debug code, or improve the language and clarity of your report, but the report itself must be written by you, not generated by AI.

Rules:

- **Allowed:** using AI to proofread, rephrase, restructure, or polish text you've written; getting help with code, debugging, or understanding concepts.
- **Not allowed:** having AI generate report content, such as sections, paragraphs, or explanations, that you then submit as your own writing.

- Every submission must include a short statement describing how AI tools were used (or “No AI tools were used,” if applicable). Be as specific as you can, for example “Used Claude to debug a recursion error in Task 2” or “Used ChatGPT to improve phrasing in Section 3.”. A general statement like “AI was used for coding help” is not sufficient.
- You must be able to explain and defend every part of your submission, regardless of how it was produced.
- Verify AI output before submitting since it can be confidently wrong.
- Any exercise where AI use is restricted further will say so explicitly.

Submissions without an AI usage statement will be treated as incomplete.

Since these exercises examine this course, we would like to ask you not to distribute or make your solutions publicly available, e.g., by putting them on a public GitHub repository. A private repository on, e.g., gitlab@liu is of course fine.

Thanks for your assistance!

4 CONTROLLER IMPLEMENTATION ENVIRONMENT

There are pre-implemented code to help with implementation. One important class is the path that the vehicle should follow. Read Appendix A carefully to understand how the class `SplinePath` works.

You define your controllers in a class, which is described in more detail in Appendix B. There are skeleton files available for both `PurePursuitController` and `StateFeedbackController`.

For example, a `PurePursuitController` object is defined as

```
1 controller = PurePursuitController(l, L, ref_path)
```

where `l` is the look-ahead horizon, `L` the car wheel base, and `ref_path` the path the controller should follow. Now that the controller object is defined, define your car object and assign your controller to the car as

```
1 car = SingleTrackModel()
2 car.controller = controller
```

Now, you can simulate your car and controller with the commands

```
1 Tend = 80
2 Ts = 0.1
3 w0 = [0, 1, np.pi / 2 * 0.9, 2] # (x, y, theta, v)
4 t, w, u = car.simulate(w0, T=Tend, dt=Ts)
```

where `w0` is the initial state, `Tend` the end time (or until the controller says stop), and `Ts` the sampling time for the controller and also the integration step. This means that the `controller.u`-method will be called every `Ts` seconds².

5 DISCUSSION TOPICS, QUESTIONS

This section summarizes a number of discussion topics and questions you should implement, investigate, and reflect upon to pass this exercise.

² You can choose sampling interval as you like, but $T_s = 0.1$ is a recommended start.

5.1 Pure-pursuit controller

Exercise 5.1. In pseudo-code, write down an approach to find the pursuit-point.

For efficiency, make sure you have a simple way of finding the pursuit-point. It is not crucial that the pursuit point is *exactly* at distance l , however, make sure you use the true distance when computing your control action.

Hint 1: Avoid using the project-function from Section A since this is a computationally expensive operation. Expect to spend no more than 40 msec (depending on your computer and implementation) in the controller for the provided scenario. If you spend significantly more time than that, reconsider your approach.

Hint 2: It is typically not a good idea to represent the look-ahead horizon circle with many points and check with (close) intersection points of the path. This typically leads to a very inefficient solution.

Exercise 5.2. In pseudo-code, write down how to find the x-coordinate, x , of the pursuit point p_p in the vehicle-local coordinate system. In particular, explain why the first element of the vector $\Delta = p_p - p_{\text{car}}$, where p_{car} are the coordinates of the car, is *not* equal to x .

Exercise 5.3. Implement a pure-pursuit controller to follow the path `ref_path` that is given by the `MiniController`. Experiment with different look-ahead horizons and initial states. Think about what happens if the vehicle is further away from the path than the look-ahead horizon. Comment on how the look-ahead horizon affects control performance and what considerations should be taken when choosing the horizon.

Skeleton files are given by `main.m` and `PurePursuitController.m`. Discuss the results. Figure 1 shows sample controller performance that you should achieve. For

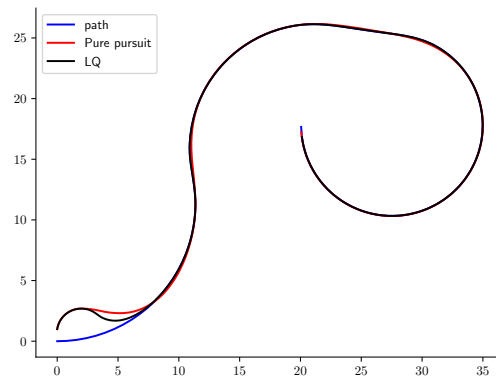


Figure 1: Sample controller performance for pure-pursuit and state-feedback (LQ) controller.

example, note that there should be *no bias* during the curves.

If you have significantly worse performance, have a closer look at your solution.

5.2 Linear state feedback controller

The next step is to explore a more classical control-oriented approach with state feedback where the path-following problem is transformed into a stability problem. The first step is to re-write the vehicle motion equations in the error frame, the Frenet frame, shown in Figure 2.

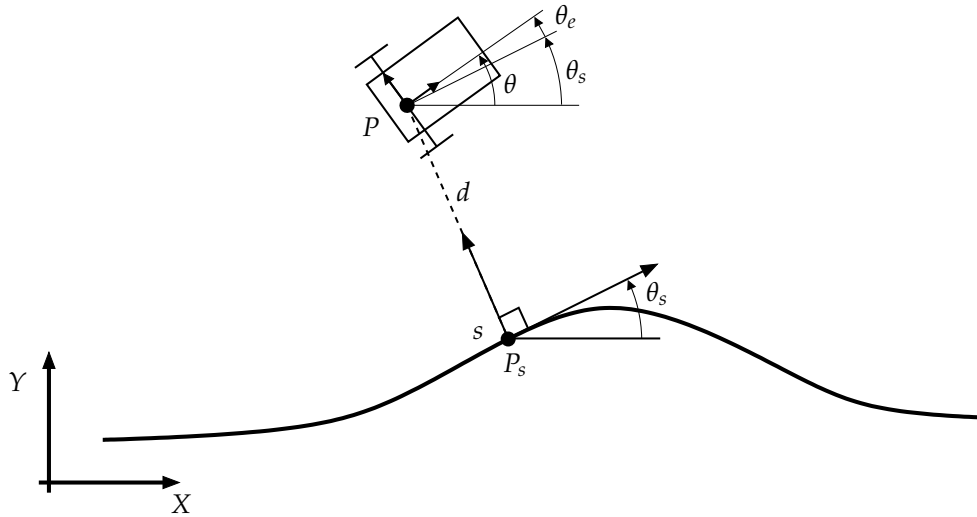


Figure 2: Frenet frame.

Exercise 5.4. Implement a linear state-feedback controller

$$u = u_0 - K \begin{pmatrix} d \\ \theta_e \end{pmatrix} \quad (1)$$

where u_0 is a feed-forward term, K the feedback gain, d the path error (distance from path), and θ_e the orientation error. Important, implement such that u is the desired *curvature*, and then transform u into a steering angle for a single-track model using

$$\tan \delta = Lu$$

Use the Frenet frame description in the last exercise to design the feedback-gain. Implement the controller in the skeleton file `StateFeedbackController.m`.

You can do a direct design of the feedback gain K or, if you would like, you can design an LQR controller. Think about a suitable choice of feed-forward term u_0 , or stated differently what is a suitable choice of control signal if you are on the path, i.e., when the error terms $d = 0$ and $\theta_e = 0$.

Experiment with your controller, similar to the pure-pursuit controller, and report your findings. In particular, vary controller gain K or the weight matrices in the LQR design, and discuss the results.

If you choose to do an LQR design, see Appendix C for a quick summary, and be sure to consult the lecture slides for comments on implementation. Do note that the equations in Appendix C are in discrete time and the linearization expression in the lecture slides is in continuous time so you need to discretize the linearization. A simple Euler forward discretization method is sufficient here.

To obtain the state-space matrices to be used in the LQR design, write down the motion equations for the single-track model in the Frenet frame from the lecture notes, i.e., write down the dynamics of the error dynamics d and θ_e . Then, linearize the error dynamics around the reference path and around a straight path, i.e., with $c(s) = 0$. This linearized model will be used in the state-feedback design.

The projection operation of the vehicle position on the path is crucial in this controller, see the `SplinePath` class described in Appendix A.

Exercise 5.5. Describe why the projection operation is difficult for a general path. Also, find an example where the projection is non-unique. Also, read through

the code for the `SplinePath` and get an understanding of how this implementation works.

Exercise 5.6. Describe and explain what happens when you vary the feedback gains or, alternatively, the weights in the LQ-controller.

5.3 Non-linear state feedback controller

The linear feedback controller should perform well in a close vicinity of the path but may experience problems when at a significant distance from the path. The exercise below is intended to illustrate such problems and introduce a non-linear feedback law that mitigates such situations.

Exercise 5.7. To illustrate the problems with a linear feedback law, start your simulation far away from the reference path, for example in the point:

$$(x, y, \theta, a) = (-5, 10, 0.9\frac{\pi}{2}, 2)$$

You should observe non-desirable behavior of your vehicle. If not, try to make your controller more aggressive, i.e., make it approach the path faster. Discuss and try to explain the observed behavior.

Exercise 5.8. Now, copy your implementation `StateFeedbackController.m` to a new file `NonlinearStateFeedbackController.m` and modify the feedback controller³ (1) to

$$u = u_0 - K \left(\frac{\sin(\theta_e)}{\theta_e} d \right) \quad (2)$$

The factor $\sin(\theta_e)/\theta_e$ has a removable singularity at $\theta_e = 0$ and can be approximated using its series expansion

$$\frac{\sin(\theta)}{\theta} = 1 - \frac{\theta^2}{6} + \frac{\theta^4}{120} - \frac{\theta^6}{5040} + \mathcal{O}(\theta^8)$$

Implement and explore the effects of this non-linear controller. Again, remember that u here is the curvature that should be transformed into a steering angle.

Comment on the properties of the control signal in the first transient part of the path and how this is related to controller tuning.

Exercise 5.9. Compare paths and control signals for the three different controllers and discuss the results.

It is required to include a plot where control signals for all controllers are plotted in the same plot for the same starting state, for example, $w_0 = (0, 1, 0.9 \cdot \pi/2, 2)$. This will highlight the properties of the controllers. Use the `path_error` method to plot the distance error as a function of time.

³ For details on this control law, see the paper Werling, M., Gröll, L., and Bretthauer, G.. "Invariant trajectory tracking with a full-size autonomous road vehicle.". IEEE Transactions on Robotics 26.4 (2010): 758-765

A SPLINEPATH

For this exercise, a helper class, `SplinePath`, is provided that represents a 2-dimensional path using spline functions. The class implements a parametric representation $p(s)$ where the path parameter $s \in [0, \text{length}]$. With this class, important operations on a path is implemented, such as

- Get coordinates for an arbitrary point on the path
- Compute the curvature for any point on the path
- Compute the heading for any point on the path
- Perform an orthogonal projection on the path
- Compute the path error for a given trajectory

To create and plot a path, let the $N \times 2$ matrix `p` represent N points on the path. Then the path object `ref_path` is created by

```
1 ref_path = SplinePath(p)
```

The path is parameterized by a parameter $s \in [0, L]$ where L is the length of the path. The coordinate of the path at position s is given by $p(s)$.

A position parameter vector `s`, with 200 points distributed equidistantly over the path length, is created below and then used to plot the path where class methods `length()`, `x()`, and `y()` are used as below to produce Figure 3(a).

```
1 s = np.linspace(0, ref_path.length, 200)
2 _, ax = plt.subplots(num=10, clear=True)
3 ax.plot(nom_path.x(s), nom_path.y(s))
4 ax.set_xlabel("x [m]")
5 ax.set_ylabel("y [m]")
```

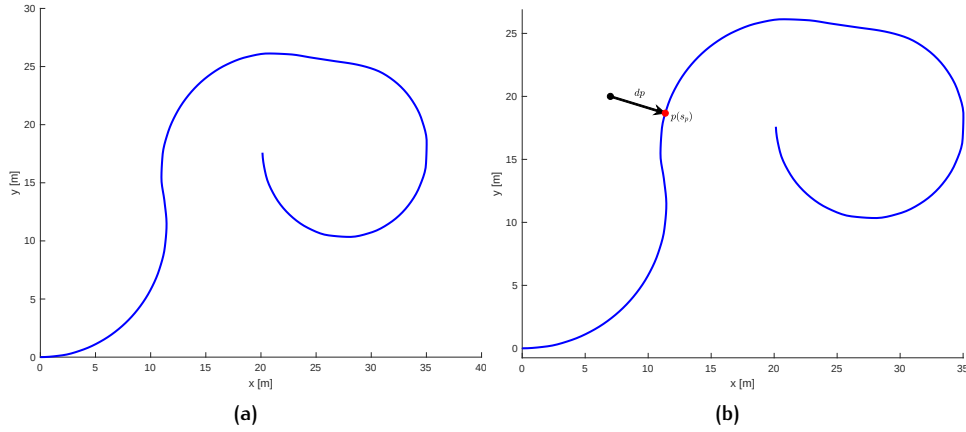


Figure 3: Figure (a) shows a sample path and figure (b) the orthogonal projection of point $(7,20)$ onto the path.

In the control algorithms, for example, when determining how large the path error is, it is important to project a given point onto the path. In general, this is a difficult problem and for the projection to be well defined the point needs to be sufficiently close to the path. Here, a numerical approximation is implemented and to project a point p_{car} call the `project`-method as

```
1 s_p, dp = ref_path.project(p_car, so)
```

where s_0 is an approximate path parameter, i.e., an approximate position on the path and the location where the search for the projection point is initiated. The projection method has additional parameters you can use to tune operation, see the help text for the method for more details. The function has default values that are appropriate for this exercise so you should not have to modify them here.

The function call returns s_p which is the path parameter for the projection, i.e., $p_{\text{projection}} = p(s_p)$, and dp is the *signed* distance between p_{car} and $p_{\text{projection}}$. Figure 3(b) illustrates the orthogonal projection and also shows dp and $p(s_p)$. To illustrate the role of the initial guess s_0 , consider Figure 4 where the black point should be projected onto the path. Since it is in between two segments of the path it is unclear where to project. The red crosses indicate two different starting points for the search, i.e., approximate positions on the path. The blue points indicate the corresponding projection points on the path. This illustration shows, in the case of

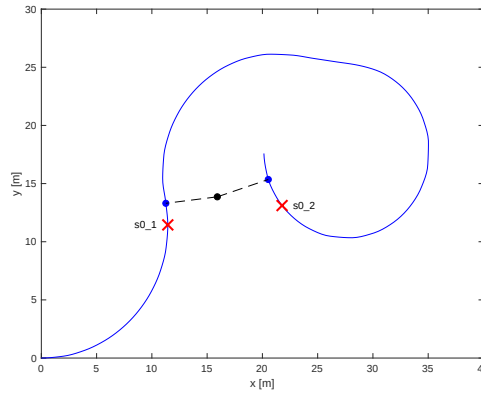


Figure 4: Influence of the initial guess of the path position when projecting.

a convoluted path, that it is important to keep track of approximate positions on the path to get consistent projections.

Another important class method is `heading` which computes a tangent and normal vector of the path at a given position. The call

```
h, nc = ref_path.heading(s)
```

returns *normalized* tangent and normal vectors, i.e., $|h| = |n_c| = 1$.

Other useful class methods are `c(s)` which returns the curvature at point s and `path_error` method can be used to compute the path error given a vehicle trajectory.

Type `help SplinePath.method_name` at the Matlab prompt for more detailed documentation.

Useful class methods: `x()`, `y()`, `length()`, `c()`, `project()`, `heading()`, `path_error()`.

B CONTROLLER OBJECTS

To implement a controller, you have to implement two functions;

- $u(t, w)$ - The main controller function called at time t in state $w = (x, y, \theta, v)$. Returns control signal (δ, a) where δ is the steering angle and a the acceleration.
- $run(t, w)$ - Returns true if the controller should run and false if the controller should stop, e.g., when reaching the goal state.

Consider a simple open-loop controller that only gives a steering angle as a function of time as

$$u = \begin{pmatrix} \delta \\ a \end{pmatrix} = \begin{cases} (10^\circ, 0) & 0 \leq t < 10 \\ (-11^\circ, 0) & 10 \leq t < 20 \\ (0^\circ, 0) & 20 \leq t < 23 \\ (-15^\circ, 0) & 23 \leq t < 40 \\ (0^\circ, 0) & t \geq 40 \end{cases}$$

The Python implementation, in `main.py`, looks like

```

1     class MiniController(ControllerBase):
2     def __init__(self):
3         super().__init__()
4
5     def u(self, t, w):
6         a = 0.0
7         if t < 10:
8             u = [np.pi / 180 * 10, a]
9         elif 10 <= t < 20:
10            u = [-np.pi / 180 * 11, a]
11        elif 20 <= t < 23:
12            u = [-np.pi / 180 * 0, a]
13        elif 23 <= t < 40:
14            u = [-np.pi / 180 * 15, a]
15        else:
16            u = [-np.pi / 180 * 0, a]
17        return u
18
19    def run(self, _t, _w):
20        return True

```

where the first method is the constructor method, which calls the constructor of the super-class `ControllerBase`. The constructor method can be used to pass parameters to the controller. For example the feedback gain K in the state-feedback controller or the look-ahead horizon for the pure-pursuit controller. In the templates provided, you do not need to change the constructor methods (but you may if you want to add additional functionality of course). The run-method determines when to end the simulation/the vehicle has reached the goal state.

Note that you can add any states, properties, or methods that you like to your controller as long as methods `u` and `run` are implemented. There are skeleton implementation provided for both `PurePursuitController` and `StateFeedbackController`.

For the `PurePursuitController`, an extended base class `PurePursuitControllerBase` provides some visualization functionality to help debug pursuit point selection.

C LINEAR QUADRATIC REGULATOR - LQR

LQR control is an optimal control technique for linear systems. Consider a time-discrete system

$$x_{t+1} = Ax_t + Bu_t \quad (3)$$

and we want to find the control signal u_t that minimizes the cost function

$$J = \sum_{t=0}^{\infty} x_t^T Q x_t + u_t^T R u_t$$

where Q and R are weighting matrices penalizing the error in x and the size of the control input u respectively. The matrices Q and R are used as tuning parameters, specifying what to prioritize; fast convergence in the states or small control signals.

In this case, there exists an explicit solution to the optimization problem. The optimal control signal is

$$u = -Kx$$

where

$$K = (R + B^T P B)^{-1} B^T P A$$

and P is the unique symmetric solution to the discrete-time algebraic Ricatti equation (DARE)

$$P = A^T P A - A^T P B (R + B^T P B)^{-1} B^T P A + Q$$

The Ricatti equation has a unique solution under general conditions, e.g., if (3) is controllable. The Ricatti equation can be solved in Matlab using the `idare` command. In python a solver `solve_discrete_are` is available in module `scipy.linalg`.

D EXTRA ASSIGNMENT

In this assignment, you will implement a Model Predictive Controller (MPC) for doing path tracking. Using a linearization strategy, you will formulate a linear MPC problem with a quadratic cost function for which there exist efficient solvers.

In this exercise, the CasADi tool for nonlinear optimization and algorithmic differentiation will be used to formulate and solve the optimization problems. See Section D.1 for instructions on how to run CasADi. For more information on MPC controllers, see lecture “*Model Predictive Control for Autonomous Vehicles*” and the recommended reading literature.

The MPC controller will be formulated in the Frenet frame, similar to the state-feedback controller from the basic exercise. The error dynamics, in the Frenet frame, with input $u = \delta$ and with state $w = (d(t), \theta_e(t))$ where $d(t)$ is the distance error and $\theta_e(t)$ the heading error. Let d_0 and $\theta_{e,0}$ be the distance and heading error at $t = t_k$, then the optimization problem to be solved in the MPC controller is in this exercise formulated as

$$\begin{aligned} \min_{u(t)} \quad & \int_{t_k}^{t_k+h_p} \gamma_d d(t)^2 + \gamma_\theta \theta_e(t)^2 + \gamma_u u(t)^2 dt \\ \text{s.t.} \quad & \dot{w} = g(w, u) \\ & d(t_k) = d_0 \\ & \theta_e(t_k) = \theta_{e,0} \\ & |u| \leq \delta_{\text{lim}} \end{aligned} \tag{4}$$

where h_p is the prediction horizon, u the steering angle, and γ_d , γ_θ , and γ_u are weighting factors for the distance error, heading error, and actuation (steer angle) size respectively. This problem is solved repeatedly and the first control output is applied at each sample instant.

There is a skeleton file `extra.{m,ipynb,py}`, where you can implement and simulate your MPC path-following controller.

Before you start: Read through the code in class `ModelPredictiveController` and ensure that you understand how it is supposed to work.

Exercise D.1. Start by implementing the cost function in the method `construct_problem`, i.e., implement

$$J = \sum_{i=0}^N \gamma_d d_i^2 + \gamma_\theta \theta_{e,i}^2 + \gamma_u u_i^2$$

which is a discrete version of the loss function in (4).

Use the CasADi function `sumsqr` to compute the sum-of-squares needed. Also, use the `heading_error` function from the state-feedback controller in the basic exercise.

Exercise D.2. Write down the Frenet equations, i.e., function g in (4). Then, linearize the equations and implement them in the method `error_model(w, v, u, κ)` in the `ModelPredictiveController` class where $w = (d, \theta_e)$ is the error state, v current velocity, and κ the current curvature. Note that the controller is written in continuous time, the controller is discretized in the controller using a Runge-Kutta integration method.

Hint: Important that you ensure that the error model is linear. Therefore, assume constant velocity, constant curvature, small θ_e ($\sin \theta_e \approx \theta_e$), and small u ($\tan \delta \approx \delta$),

and path velocity equal to car velocity ($\dot{s} \approx v$). For more information about the linearization procedure, see details in lecture “*Ground vehicle motion control*”.

Exercise D.3. Now, you should have everything you need to run the MPC path-following controller. Make a first experiment and see that it works. When building the optimization problem, you get a printout similar to

1	This is casadi::QRQP	
2	Number of variables:	302
3	Number of constraints:	302

Note that the number 302 may be different for you since it depends on the parameterization of your controller.

Explain how the number of variables and constraints in the optimization problem corresponds to controller parameters such as the sampling period and prediction horizon. Also, determine how many of the constraints are equality constraints and how many are inequality constraints. Comment on what that means for the degrees of freedom in the optimization problem.

Exercise D.4. Now, experiment with different initial states, values for controller parameters (as defined above); the sampling rate (dt), prediction horizon (h_p), steer limitations (steer_limit) as well as the different weights (gamma_d, gamma_theta, gamma_u).

Find illustrative experiments and discuss the results related to tracking performance, computational time, and quality of the performed maneuver. In particular, make sure to validate that controller constraints on control signals are not violated.

Exercise D.5. Discuss how the properties of the MPC controller compare with the state-feedback controller from the basic exercise, advantages/disadvantages.

D.1 CasADi installation

CasADi is pre-installed in the student labs, you only need to add the installation directory to the Matlab path. You do this by including the line below in your scripts

1	<code>addpath /courses/tsfs12/casadi</code>
---	---

For Python users, CasADi is installed in the provided virtual environment that is activated by

1	<code>% source /courses/tsfs12/env/bin/activate</code>
---	--

To run on your own computer, download and install CasADi according to the instructions on <https://web.casadi.org/>.

In Python, you install the CasADi package by running

1	<code>pip install casadi</code>
---	---------------------------------

after you have activated your virtual environment.